



KUBESPECTRA

OPEN SOURCE INNOVATION – SECURE BY DESIGN

Kyverno Deep Dive: Debugging, Testing and Monitoring

Marie Padberg

11.09.2025

Why KubeSpectra



We build, implement and develop Kubernetes-based container platforms.



Containerization



DevOps



Migration

We enable the quick, secure and flexible use of applications in the cloud.

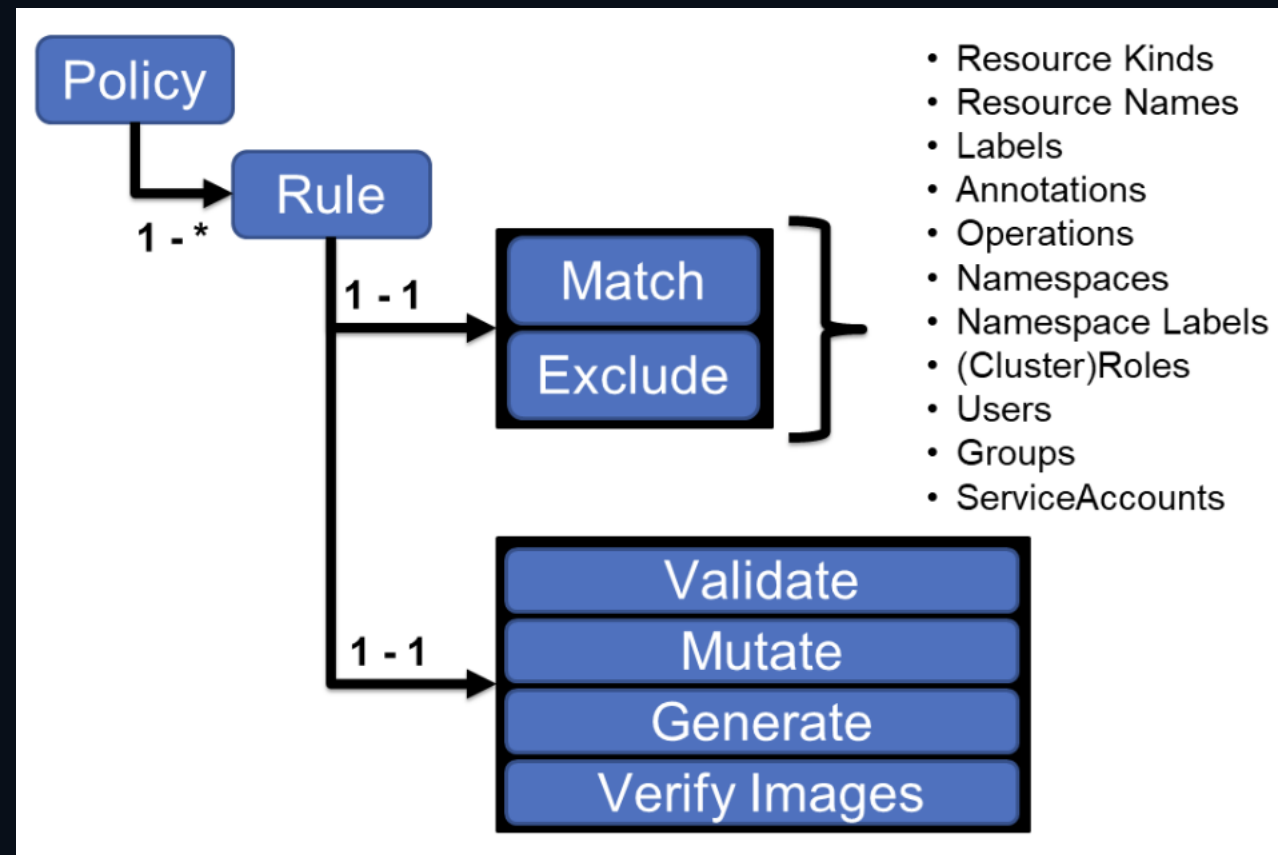
Agenda

1. Short Kyverno recap
2. Implementation of a Kyverno validation policy
3. Debugging and testing this policy
4. Monitoring the implemented policy

Kyverno Recap - Policy Engine



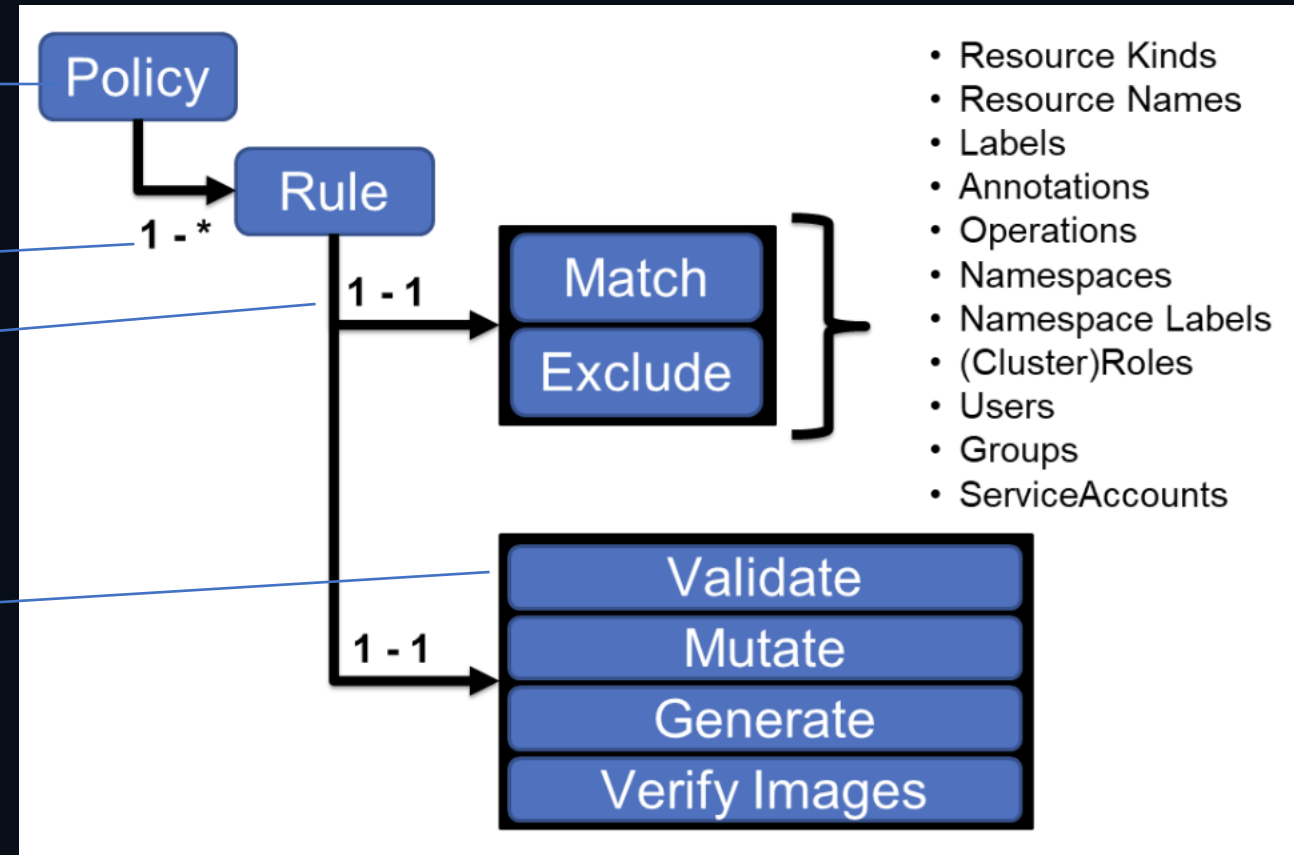
- A Kyverno policy is a collection of rules
- Each rule contains a *match* clause and possibly an *exclude* clause
- Scope: cluster-wide or namespace-wide
- 2 modes:
 - **Audit**: violations are recorded but allowed
 - **Enforce**: violations lead to rejection



Kyverno Recap - Beispiel Validation Policy



```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
name: validate-image-tag
spec:
  rules:
  - name: deny-latest-tag
    match:
      any:
      - resources:
          kinds:
          - Pod
    validate:
      failureAction: Audit
      message: "Using the mutable image tag
'latest' is not allowed."
      pattern:
        spec:
          =(containers):
          - image: "!*:latest"
```



A new Policy is needed...



- Requirement: Ensure that namespaces have a projectId label that matches the expected project ID from a predefined project-to-namespace mapping, excluding the default-namespace.

```
metadata:  
  name: team-x  
  labels:  
    projectId: pid12345
```

- Prerequisite: We know the mapping between projectId and name
- Implementation:
 - Policy must know the mapping between projectId and name
 - Namespaces without or with incorrect projectId should be rejected
 - default-namespace needs an exception



What is JMESPath?

- JMESPath (pronounced „James path“)
- Declarative query language for JSON data
- Enables extraction & filtering of data structures

Why is this important in Kyverno?

- Indispensable in preconditions, deny-rules or foreach-loops
- Structure verification through pattern is not always sufficient



Typical use cases

- Access to labels, annotations, container images etc.
- Enables complex conditions: *request.object.metadata.labels.app == 'frontend'*
- Iterations & filters: *spec.[initContainers, containers][].image*
- Message field if a policy blocks a resource

Syntax short overview

- Always `{{ }}` for JMESPath expressions
- Only `"{{...}}"`, if the field is only the expression, e.g. label - *appns: "{{ request.namespace }}"*
- Expression + Text → Quotes optional, e.g. message: *"Namespace is {{ request.namespace }}"*
- `.` =sub-expression → access nested fields



What does `{{ request.namespace }}`?

1. Create a pod: `kubectl run nginx --image=nginx -n psa-test`
2. Kubernetes API-Server processes the request
 - a) Validates the YAML-file
 - b) Adds system default values (e.g. Timestamps)
 - c) Packages the request into an AdmissionReview object and sends it to the Admission Controllers
3. The AdmissionReview object contains among others
 - a) The origin resource
 - b) Information about the action (CREATE, DELETE,...)
 - c) Namespace
 - d) Who triggered the request
 - e) ...

JMESPath – AdmissionReview



- Example from an AdmissionReview object

```
{
  "kind": "AdmissionReview",
  "apiVersion": "admission.k8s.io/v1",
  "request": {
    "name": "nginx",
    "namespace": "psa-test",
    "operation": "CREATE",
    "userInfo": {
      "username": "kubernetes-admin",
    },
  },
  "object": {
    "kind": "Pod",
    "apiVersion": "v1",
    "metadata": {
      "name": "nginx",
      "labels": {...}
    },
  },
}
```

JMESPath – Custom Filters



- Kyverno extends standard JMESPath filters to solve typical Kubernetes use cases
- `contains()`-filter
 - Returns *true* if the given *\$subject* contains the given *\$search string*
 - Example: *contains('foobar', 'foo')* -> Result: *true*
 - Useful to check whether a specific element or substring is present
- `pattern_match()`-filter
 - Compares simple, non-regex expression by specifying an input pattern and the string/number to compare
 - Output is a Boolean
 - Useful when simple comparisons are to be performed, typically with strings or numbers

Kyverno CLI



- Useful to validate and test the behavior of policies
- *jp* is a subcommand of the Kyverno CLI
 - Enables testing of JMESPath expressions
- List of available custom JMESPath filters from Kyverno: `kyverno jp function`
- More examples:
 - `jp` → entry into "JMESPath"-mode
 - `query` → executes the JMESPath expression
 - `<JMESPath-expression>` → filter or access path in the JSON document

```
kubectl get pod nginx -o json | kyverno jp query 'spec.containers[0].image'
```

```
kyverno jp query -i admissionreview.json 'request.namespace'
```

Our Policy: validate-ns-label-projectId



- *Requirement: Ensure that namespaces have a projectId label that matches the expected project ID from a predefined project-namespace-to mapping, excluding the default-namespace.*
- Required JMESPath expressions:

```
request.object.metadata.labels bzw. request.object.metadata.labels.projectId  
request.object.metadata.name
```




Kyverno Validation Policies – Variables

- Make policies dynamic by enabling references to data
- Predefined variables
 - `system:serviceaccount:<namespace>:<user>`
- Policies can reference themselves, e.g. with `$(./../..)`, to reuse values from the policy structure (similar to a relative path)
 - Example: `livenessProbe.port` is set to the same value as `readinessProbe.port` via `$(./../..../readinessProbe/tcpSocket/port)`
- Include ConfigMaps, Kubernetes API server calls, other cluster services or image-registries as variable via *context*, e.g.
 - Variables from ConfigMaps: access within the policy using `{{<context-name>.data.<key>}}`

Our Policy: validate-ns-label-projectId



- *Requirement: Ensure that namespaces have a projectId label that matches the expected project ID from a predefined project-namespace-to mapping, excluding the default-namespace.*
- Use a ConfigMap for the policy:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: project-namespace-mapping
  namespace: kyverno
data:
  pid12345: team-x
  pid67890: team-y
```

```
context:
- name: projectMap
  configMap:
    name: project-namespace-mapping
    namespace: kyverno
```



Kyverno Validation Policies – Deny Rules

- If a deny condition evaluates to true, the resource is blocked (admission request rejected)
- Inside a deny blocks:
 - **all** vs. **any**
 - **key**: JMESPath path or variable access (e.g. request.object.spec...)
 - **operator**: comparisons like Equals, NotEquals, LessThan, AnyNotIn, ...
 - **value**: comparison value, also list types
 - **message**: optional error message – displayed in Admission-Response

Difference from pattern validation:

- **Pattern** (Overlay): required structure / values → fail if not matching pattern
- **Deny**: reject on matching conditions → fail if conditions true

Our Policy: validate-ns-label-projectId



- *Requirement: Ensure that namespaces have a projectId label that matches the expected project ID from a predefined project-namespace-to mapping, excluding the default-namespace.*
- Required conditions:

```
deny:
  conditions:
    any:
      - key: "{{ request.object.metadata.labels.projectId | | " }}"
        operator: Equals
        value: ""
        message: "The label projectId is missing"
      - key: "{{
          pattern_match(
            '{{projectMap.data.{{request.object.metadata.labels.projectId}}}',
            '{{request.object.metadata.name}}'
          )
        }}"
        operator: Equals
        value: false
        message: "Namespace has the wrong ProjectId."
```



Kyverno Validation Policies – Preconditions

- Control when a rule is executed
- Evaluate after match/exclude, but before the actual rule body
- If precondition evaluates to false, the rule body is skipped
- Since Preconditions often check fields that are optional, it may be necessary to use a JMESPath syntax for non-existence checks like (`|| ''`)
- Can be extended with context data

Our Policy: validate-ns-label-projectId



- *Requirement: Ensure that namespaces have a projectId label that matches the expected project ID from a predefined project-namespace-to mapping, excluding the default-namespace.*
- Exception for default-Namespace:

```
preconditions:  
  all:  
    - key: "{{ request.object.metadata.name }}"  
      operator: AnyNotIn  
      value: "default"
```

Testing





Testing – Manually with CLI

```
kyverno test /path/to/folderContainingTestYamls
```

- Tests a specific set of resources against one or more policies, to compare desired results (specified in advance in a separate test manifest file) with actual results
- *kyverno test* looks for a file *kyverno-test.yaml* and executes the tests defined in it
- In the tests, you can decide between 4 options (pass, skip, fail, warn). If the result of the test gives the desired option, it is considered passed

Testing – Test Structure



- Snippet from *kyverno-test.yaml*

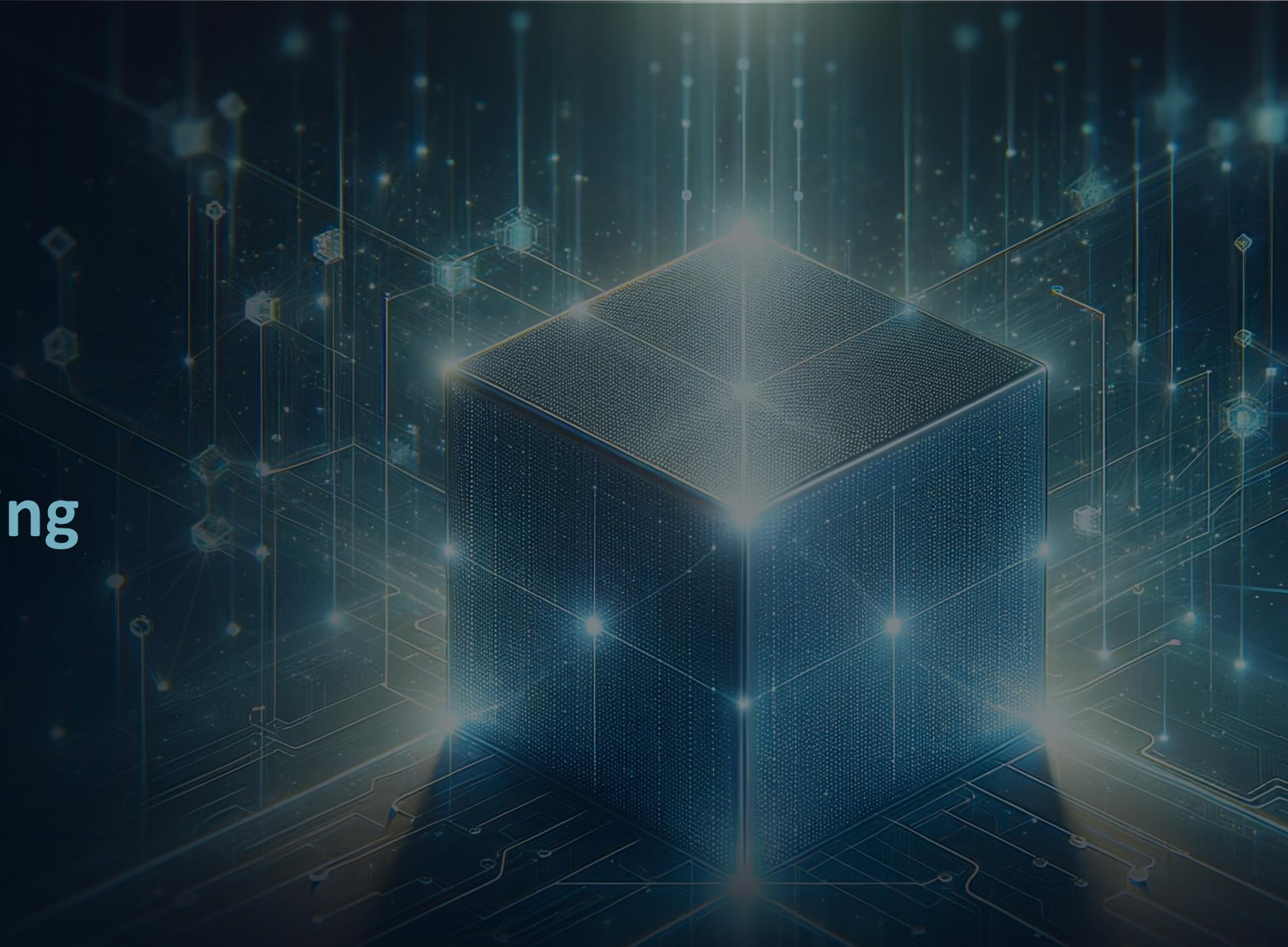
```
apiVersion: cli.kyverno.io/v1alpha1
kind: Test
metadata:
  name: validate-namespace
policies:
  - ../../policies/validate-namespace.yaml
resources:
  - namespace-success.yaml
  - namespace-fail.yaml
variables: variables.yaml
results:
  - policy: validate-namespace-projectid
    rule: validate-ns-label-projectid
    resources:
      - team-x
    kind: Namespace
    result: pass
```

Testing – Automation with CI



- Two main use cases
 1. Development teams are responsible for their manifests and want to check compliance in a pull request before deployment takes place
 - Pipeline only successful if all tests pass
 - Tests failing here will also fail in the Kubernetes cluster
 2. Testing the rules
 - Verify behavior of policies
 - Ensure expected behavior after changes
 - Test new Kyverno versions
- Implementation using Github Action: [kyverno/action-install-cli@v0.2.0](#)
- The Git repository must contain the policies, the resources to be tested and the corresponding tests

Reports & Monitoring

A glowing blue cube with a grid pattern, surrounded by a complex network of lines and nodes, symbolizing data and technology.

Reports - Example Validation Policy



```
$ kubectl create deployment busybox --image=busybox:latest -n psa-test
```

```
$ kubectl get policyreport -n psa-test
```

NAME		KIND		NAME		PASS	FAIL	WARN	ERROR
SKIP	AGE								
43c9a204-37a4-4960-ba6b-c974ee29fb6d		Pod		deployment-check-example-7f854fdc76-					
zrdxz	0	1	0	0	0				99s

```
apiVersion: wgpolicyk8s.io/v1alpha2
kind: PolicyReport
results:
- message: 'validation error: Using the mutable image tag "latest" is not allowed.
  rule deny-latest-tag failed at path /spec/containers/0/image/'
  policy: validate-image-tag
  result: fail
  rule: deny-latest-tag
  scored: true
  source: kyverno
scope:
  apiVersion: v1
  kind: Pod
  name: deployment-check-example-7f854fdc76-zrdxz
  namespace: psa-test
summary:
  error: 0
  fail: 1
  pass: 0
  skip: 0
  warn: 0
```

Reports – Policy Reporter



- Policy Reports
 - Distributed across all namespaces
 - Lack of clarity in reports, as many results are in one report
 - Difficult to find failed rules
- Policy Reporter
 - Developed to make results of Kyverno validation policies more visible and observable
 - Also provides a standalone dashboard to get a graphical overview of all results and information about Kyverno policies
 - Optional metrics endpoint can be used to monitor violations in tools like Grafana

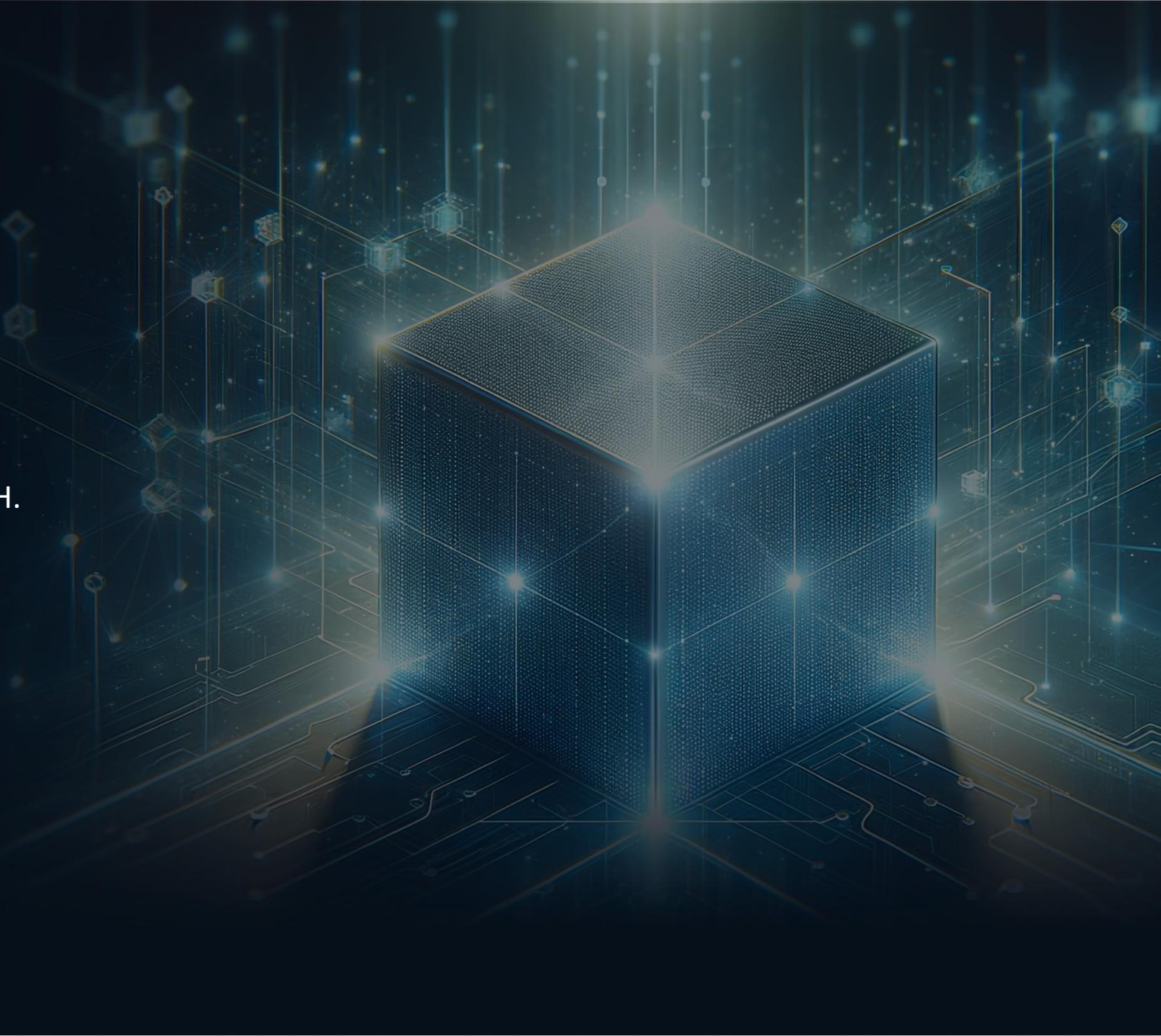
Conclusion



- Understanding JMESPath and know about the AdmissionReview object is crucial
- CLI for testing and experimenting
- Policy Reporter to get a quick overview
- Many ways lead to Rome: Our policy can also be written differently, e.g. without `pattern_match()`, or move precondition into deny block, ...
- Only a subset of popular features shown – Kyverno offers many more

LinkedIn





KubeSpectra is a brand of infologistix GmbH.
All rights reserved.

infologistix GmbH
Gutenbergstraße 7
85748 Garching
Germany

www.kubespectra.io / www.infologistix.de
kubespectra@infologistix.de
T +49 89 81885979